Guia de calibração da câmera

Correção de pose v2 (bordas das caixas, tilt para cima e para baixo) · `maze_solver`, branch `with_camera_v2`

Este guia mostra, na ordem, como calibrar a câmera do kart com o `camera_calibrate.py` e como conferir o resultado antes de usá-lo no runner. A ferramenta fala direto com o kart (vídeo MJPEG e comandos de servo por HTTP), sem CoppeliaSim e sem o runner aberto. Tudo o que ela mede vai para um único arquivo, `camera_link/camera_calib.json`, que o runner lê sozinho.

> **O que a câmera v2 faz**
> Em cada pan (frente, direita, esquerda, nessa ordem) a cabeça olha **para cima** (topos das caixas, tilt "cima") e **para baixo** (base das caixas no chão, tilt "baixo"). Cada borda vista é comparada com a mesma borda na simulação do labirinto. Na frente, as bordas se dividem em metade esquerda e metade direita: as duas deslocadas para o mesmo lado indicam **yaw**, e as duas se afastando ou se aproximando indicam erro **longitudinal**. O yaw é corrigido girando o kart, e a longitudinal só move a âncora; depois a câmera mede de novo. O objetivo não é precisão: é impedir que o erro que sobra da troca de referência acumule numa corrida longa até o kart encostar numa parede.

Passo	O que faz	Quando	Resultado
1	Confere câmera, servidor e branch	sempre	kart pronto
2	`intrinsics`: focal e distorção da lente, com tabuleiro	uma vez	`fx`, `fy`, `cx`, `cy`, `dist`
3	Medidas da montagem com régua	uma vez	posição da lente
4	Posicionar o kart numa célula boa	antes de 5 a 7	pose conhecida
5	`extrinsics`: ajusta tilts, vistas laterais e altura	quando a cabeça muda	calibração salva (com backup)
6	`check`: mede a pose e gera imagens	sempre	PNGs + números
7	Validar em outra célula	depois de 5	`check` coerente
8	Usar no runner	—	correção nas células boas

1. Antes de começar

1. **Branch certo.** A câmera v2 só existe no `with_camera_v2`:

   ```
   git checkout with_camera_v2
   ```

2. **Câmera funcionando no Pi.** Com o servidor do kart parado, rode no Pi `rpicam-hello -t 5000 --nopreview`. `Camera frontend has timed out` é o flat cable.
3. **Servidor do kart rodando.** Pegue o IP no OLED e abra `http://IP:8080/video` no navegador; tem que aparecer imagem.
4. **Bateria carregada.** Com a bateria fraca os servos param em posições diferentes, e a calibração sai errada.

5. **Labirinto montado** igual ao `tcc.maze` (mapa na seção 4).

Todos os comandos rodam da pasta `orquestrator`. O IP padrão é o `KART_HOST` de `maze_solver/runner/constants.py` (`192.168.0.238`); se for outro, acrescente `-i IP` logo depois de `camera_calibrate.py`.

2. Intrínsecos da lente (`intrinsics`)

Mede a distância focal real e a distorção. Basta uma vez; depois, **não gire a lente** (ela é de rosca, e o foco muda a focal).

1. Cole o tabuleiro numa superfície **rígida e plana** e meça o lado de uma casa, em metros (o seu: `0.033`).
2. Rode:

```
python camera_calibrate.py -i IP intrinsics --board 7x7 --square 0.033
```

   `--board` conta os **cantos internos**: 8×8 casas são 7×7.
3. Na janela: **ESPAÇO** captura (cantos desenhados), **C** calibra (10 capturas ou mais), **Q** sai. Faça de 15 a 20 capturas variadas: perto, longe, nos cantos e inclinado.
4. Confira: `rms` abaixo de 0,5 px; `fx` e `fy` perto de 380–400; `cx` e `cy` perto de 200 e 150.

3. Medidas da montagem (régua)

O modelo precisa saber onde fica a lente em relação ao **centro do kart**, que é o ponto que a odometria acompanha. Os padrões estão em `maze_solver/runner/constants.py`; o `camera_calib.json` tem prioridade sobre eles (`lens_xyz`, `pan_arm_m`).

O que medir	Constante	Padrão
Lente à frente do centro do kart	`CAMERA_LENS_FORWARD_M`	0,13 m
Lente para o lado (esquerda positivo)	`CAMERA_LENS_LATERAL_M`	0,0 m
Altura da lente até o chão	`CAMERA_LENS_HEIGHT_M`	0,10 m
Do eixo do servo de pan até a lente	`CAMERA_PAN_ARM_M`	0,025 m

> **A altura pode ser ajustada pelo `extrinsics`**
> Com `--altura`, o `extrinsics` estima a altura junto com o resto: os topos e as bases das caixas, vistos juntos, separam altura de inclinação. Na calibração de 05/11 ele deu **14,0 cm**, contra 10 cm do padrão. Vale medir com régua, com a cabeça no tilt de baixo, para confirmar; se a régua discordar muito, ponha o valor medido em `lens_xyz` e rode o `extrinsics` sem `--altura`.

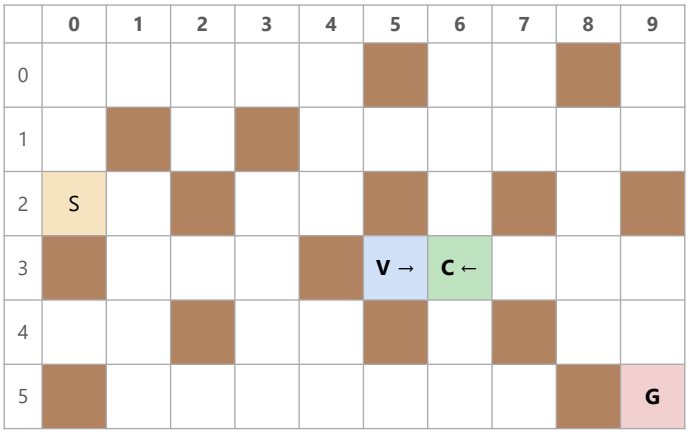
4. Posicionar o kart

O `extrinsics` e o `check` comparam o que a câmera vê com o que o mapa prevê para uma pose **informada por você**. O `extrinsics` estima o seu erro de posicionamento à parte (até uns 2 cm e 2°), mas quanto melhor a colocação, melhor a calibração. Uma cruz de fita no centro da célula ajuda.

Use uma **célula boa**, a mesma regra que o runner usa para escolher onde medir:

- a célula da frente **livre** (a lente fica 13 cm à frente do centro; uma caixa colada na frente encheria a imagem);
- **pelo menos 2 caixas à frente**: nos lados da célula da frente ou nas três células duas à frente;
- **corredor**: caixas dos dois lados do kart ou da célula da frente.

O `check` avisa quando a célula não serve (`[WARN] celula ruim para a camera v2`).



`--heading`	**Kart aponta para**
`0`	direita (leste) →
`90`	cima (norte) ↑
`180`	esquerda (oeste) ←
`270`	baixo (sul) ↓

caixa
C ← calibração:
`--cell 3 6 --heading 180`
V → validação:
`--cell 3 5 --heading 0`
S início do `tcc.maze`: célula ruim para a câmera
G goal

5. Extrínsecos (`extrinsics`)

Com o kart posicionado (leva uns 5 minutos):

```
python camera_calibrate.py -i IP extrinsics --cell 3 6 --heading 180 --altura
```

A cabeça passa pelas 6 vistas (frente, direita e esquerda, cada uma em cima e embaixo). A ferramenta ajusta:

Parâmetro	**O que é**
`pitch_cima`, `pitch_baixo`	inclinação real de cada posição de tilt
`pan_direita`, `pan_esquerda`	onde o servo de pan realmente para em cada lateral
`tilt_direita`, `tilt_esquerda`	quanto a inclinação de cada lateral difere da frente
`altura`	altura da lente (só com `--altura`)
`kart_yaw`, `kart_long`, `kart_lat`	o seu erro de posicionamento: **não é salvo**, só mostrado

São três rodadas. As duas primeiras começam com uma busca por vista, larga (±35° no tilt, ±15° nos pans); a terceira refina. Cada rodada segmenta as imagens de novo com a calibração da anterior. A saída termina assim:

```
  pitch_cima: +11.92 +- 0.19 deg
  pitch_baixo: +3.42 +- 0.26 deg
  ...
  altura: +1.90 +- 0.14 cm
  kart_yaw: -2.09 +- 0.14 deg
  kart_long: -1.17 +- 0.09 cm
[INFO] posicao estimada do kart: long -1.2 cm, lat +1.4 cm, yaw -2.1 deg ...
[INFO] calibracao anterior em ...\camera_calib.json.bak-20261105_233529
[INFO] pitch por tilt: cima -14.31, baixo +33.22; altura da lente 14.0 cm; salvo em ...
```

> **Backup automático**
> Antes de gravar, o arquivo anterior é copiado para `camera_calib.json.bak-<data>`. Para voltar, basta copiar o backup por cima do `camera_calib.json`.

> **NAO salvei: ...**
> A ferramenta **recusa gravar** quando um parâmetro termina na borda da busca, fica indeterminado (desvio grande), quando sobram menos de 4 vistas ou quando o custo não cai. Olhe as imagens `*_extrinsics_sim_<vista>.png` para entender o motivo (seção 6) e rode de novo; `--forcar` grava mesmo assim. Se a `posicao estimada do kart` vier com mais de 3 cm ou 3°, reposicione o kart, ou confira a posição da lente (seção 3).

As imagens `*_extrinsics_sim_<vista>.png` têm três painéis: a imagem real, a simulada com a calibração nova e a real com o contorno das caixas **antes** (vermelho) e **depois** (ciano). O ciano deve cair em cima das bordas reais: base das caixas nas vistas de baixo, topo nas de cima.

> **O tilt de cima fica gravado no arquivo**
> O `camera_calib.json` guarda em `tilts` a posição do tilt de cima (`tilt_pct`, hoje 8) e a inclinação medida nela. Mudar `CAMERA_V2_TILT_UP_PCT` não afeta um arquivo que já tem esse valor: edite `tilt_pct` no arquivo, apague o `pitch_deg` desse tilt (deixe `null`) e rode o `extrinsics` de novo.

6. Conferir (check)

```
python camera_calibrate.py -i IP check --cell 3 6 --heading 180
```

Faz **uma** medida completa da v2 na pose informada, sem aplicar nada e sem girar o kart:

```
[INFO] medido, 1 medida(s) | ultima: yaw -2.72+-0.12 deg, long -1.1+-0.0 cm,
       lat +1.3 cm (611 pts, rms 2.1 px)  (11.5 s)
       direita_baixo: 140 pts, desvio (-4.5, -6.8) px, resto 0.3 px
       frente_dir_baixo: 74 pts, desvio (-13.1, -2.5) px, resto 3.4 px
       frente_esq_baixo: 68 pts, desvio (-9.2, -4.9) px, resto 1.7 px
       frente baixo: comum -11.2 px (yaw), afastamento -2.0 px (longitudinal)
       grupo direita_cima fora: fora do modelo (7.7 px)
```

Na saída	Significado
`yaw`, `long`	quanto o kart real está fora da pose informada; é isso que o runner corrigiria
`lat`	estimado, mas **não aplicado** (é da referência lateral)
grupo `frente_esq_*`, `frente_dir_*`	metades esquerda e direita da vista da frente
`desvio (x, y)`	quanto as bordas reais do grupo estão deslocadas da simulação, em pixels
`comum`	deslocamento igual nas duas metades: **yaw**
`afastamento`	metades se afastando (+) ou se aproximando (−): **longitudinal**
`grupo ... fora`	grupo que não bate com o resto (pessoa, objeto, cor) e foi ignorado

Com o kart bem posicionado, `yaw` e `long` ficam perto da `posicao estimada do kart` que o `extrinsics` mostrou no mesmo lugar: é o seu posicionamento, não erro da câmera.

As imagens `*_bancada_v2_<vista>_it1.png` mostram a borda real das caixas (**verde**), a simulada na pose informada (**vermelho**) e a simulada na pose estimada (**ciano**). As `*_bancada_sim_<vista>.png` trazem o mesmo trio de painéis do `extrinsics`.

O que aparece	Causa provável e o que fazer
ciano em cima do verde	tudo certo
deslocado por igual para cima ou para baixo numa vista	inclinação daquele tilt; rode o `extrinsics`
deslocado de lado numa vista lateral	pan daquela lateral; rode o `extrinsics`
verde marcando uma emenda ou faixa do piso	o piso tem faixa com cor de papelão; o modelo apaga faixas finas, mas faixas largas atrapalham: cubra-as
modelo espelhado entre esquerda e direita	`CAMERA_PAN_LEFT_IS_HIGHER` invertido

7. Validar

1. `check` de novo na mesma célula: os números devem repetir (±0,5° e ±0,5 cm).
2. Leve o kart para a **outra** célula boa, com outro heading, e rode o `check`:

   ```
   python camera_calibrate.py -i IP check --cell 3 5 --heading 0
   ```

   Com o kart bem posicionado, `yaw` e `long` devem ficar pequenos (poucos graus, 1–2 cm).
3. Se a segunda célula não fechar, rode o `extrinsics` nela e fique com a calibração que funciona nas duas. Para comparar versões, use `--calib`:

   ```
   python camera_calibrate.py -i IP --calib camera_link/calib_teste.json extrinsics --
   cell 3 5 --heading 0 --altura
   ```

8. Usar no runner

O runner lê o `camera_link/camera_calib.json` sozinho (`--camera-calib` para outro arquivo, `--no-camera-fix` para desligar a câmera).

- **Quando roda:** fica devida a cada `CAMERA_V2_EVERY_N_CELLS` células (3). Cada célula é avaliada virada para onde o kart **chega** e, se não servir, para onde a **próxima perna sai** (o kart faz antes o giro que a perna faria de qualquer jeito). Se a célula atual for boa (seção 4), roda ali, depois da recentragem e da referência lateral. Senão, procura a primeira célula boa no plano até `CAMERA_V2_MAX_NEXT_BEST_CELLS` (8) à frente e vai **uma célula por vez** até ela, como o `NEXT_BEST_CLOSEST_AS_REFERENCE`. Sem célula boa nas 8, **mede ali mesmo**, virada no rumo menos ruim. Cada decisão aparece no terminal (`[INFO] camera v2 ...`).
- **Referência lateral:** se o lado para onde ela gira é o rumo da próxima perna, o kart fica virado para lá em vez de voltar (`CLOSEST_OBSTACLE_KEEPS_NEXT_HEADING`).
- **O laço:** mede; o **yaw** é corrigido girando o kart de volta ao eixo do labirinto (antes, a crença recebe o rumo medido, senão a próxima perna desfaria o giro); a **longitudinal** só move a âncora. Mede de novo, no máximo 2 vezes, e aceita abaixo de 1° e 1 cm.
- **Início:** logo depois do `ok`, se a célula inicial for boa.
- **`camfix`** (no prompt do runner): uma medida sem aplicar nada.
- **Registro:** um CSV por corrida em `maze_solver/logs/`.

Constante (`constants.py`)	Valor	Para quê
`CAMERA_V2_ENABLED`	`True`	`False` volta para a câmera v1 (só contatos com o chão)

Constante (constants.py)	Valor	Para quê
CAMERA_V2_EVERY_N_CELLS	3	intervalo entre medidas
CAMERA_V2_MAX_ITERATIONS	2	medidas por célula
CAMERA_V2_ACCEPT_YAW_DEG / _LONG_M	1,0° / 1 cm	aceita e para
CAMERA_V2_MAX_YAW_STEP_DEG / _LONG_STEP_M	3° / 1 cm	maior passo por medida
CAMERA_V2_MIN_TURN_DEG	0,8°	giro menor que isso não é feito (só a crença muda)
CAMERA_V2_YAW_SIGN	1.0	sinal do passo de yaw; troque se aparecer divergiu

9. Problemas comuns

Mensagem ou sintoma	O que fazer
sem conexao com IP:8080	IP errado ou servidor parado; confira o OLED e passe -i
celula ruim para a camera v2	escolha uma célula boa (seção 4)
parede e chao com cores parecidas demais	a vista foi descartada; numa calibração muito errada isso é normal na rodada 1 e se resolve nas seguintes
NAO salvei: ...	veja a seção 5; olhe as imagens e rode de novo
posicao estimada do kart grande	reposicione o kart; se repetir, confira lens_xyz
divergiu no runner	o yaw cresceu depois do passo: troque CAMERA_V2_YAW_SIGN ou recalibre
números diferentes a cada extrinsics	bateria fraca, cabeça frouxa ou kart mexido; o servo de pan anda aos saltos de 10°
a cabeça olha bem mais para baixo do que antes	o suporte mudou; rode o extrinsics de novo (foi o que aconteceu em 05/11)

10. Referência rápida

Opção	Padrão	Significado
Gerais (vêm antes do subcomando)		
`-i, --ip`	`192.168.0.238`	IP ou nome do kart
`-p, --port`	`8080`	porta do servidor do kart
`--calib`	`camera_link/camera_calib.json`	arquivo lido e gravado
`intrinsics`		
`--board`	`9x6`	cantos internos (8×8 casas = `7x7`)
`--square`	`0.025`	lado de uma casa, em metros
`check` e `extrinsics`		
`--maze`	`tcc.maze`	labirinto montado
`--cell`	obrigatório	linha e coluna do kart
`--heading`	obrigatório	0 leste, 90 norte, 180 oeste, 270 sul
`--v1`	—	versão antiga (um tilt, contatos com o chão)
só `extrinsics`		
`--altura`	—	ajusta também a altura da lente
`--span`	`15`	busca ± graus dos pans e vieses laterais
`--span-tilt`	`35`	busca ± graus da inclinação de cada tilt
`--step`	`1.0`	passo da grade, em graus
`--passes`	`8`	passadas do ajuste fino
`--forcar`	—	grava mesmo quando a ferramenta recusaria

Calibração de 05/11/2026 (célula 3 6, heading 180):

Valor	Medido
inclinação no tilt de baixo (62)	33,2° para baixo
inclinação no tilt de cima (8)	14,3° para cima
altura da lente	14,0 cm
lateral direita (pan / tilt)	−7,9° / −6,2°
lateral esquerda (pan / tilt)	+14,3° / −0,6°

Sequência completa, para copiar (troque `IP` pelo do kart):

```
git checkout with_camera_v2
python camera_calibrate.py -i IP intrinsics --board 7x7 --square 0.033
python camera_calibrate.py -i IP extrinsics --cell 3 6 --heading 180 --altura
python camera_calibrate.py -i IP check      --cell 3 6 --heading 180
python camera_calibrate.py -i IP check      --cell 3 5 --heading 0
```